

Demo - Support Vector Machine

In this demo, you will learn how to apply SVM algorithm in Python using scikit-learn library

Problem Statement

Given details of loan application predict the loan will be granted or not.

Importing Required Libraries

```
In [10]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import RobustScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
```

Dataset

Dataset includes details of different loan applications along with the application status:

- **Loan_ID:** LP001002(Application ID)
- **Gender:** Male/Female
- **ApplicantIncome:** Monthly income of the applicant
- **LoanAmount:** Amount to be loaned
- **Loan_Status:** Either the application has been accepted or not(Y/N)

• Checking Data

```
In [11]: loan_data=pd.read_csv('loan_grant.csv')
```

```
In [12]: loan_data.head()
```

Out[12]:

	Loan_ID	Gender	ApplicantIncome	LoanAmount	Loan_Status
0	LP001002	Male	5849	NaN	Y
1	LP001003	Male	4583	128.0	N
2	LP001005	Male	3000	66.0	Y
3	LP001006	Male	2583	120.0	Y
4	LP001008	Male	6000	141.0	Y

In [13]: *#Dataset shape to get number of rows and columns*
`loan_data.shape`

Out[13]: (614, 5)



Number of rows and columns in the dataset



• Checking Datatypes

In [14]: `loan_data.dtypes`

Out[14]:

Loan_ID	object
Gender	object
ApplicantIncome	int64
LoanAmount	float64
Loan_Status	object
dtype:	object

• Checking Null Values

In [15]: *#Checking for null values*
`loan_data.isnull().sum()`

Out[15]:

Loan_ID	0
Gender	13
ApplicantIncome	0
LoanAmount	22
Loan_Status	0
dtype:	int64

In [16]: `loan_data.isna().sum()`

Out[16]:

Loan_ID	0
Gender	13
ApplicantIncome	0
LoanAmount	22
Loan_Status	0
dtype:	int64



Some of the columns have null data



• Treating Null values

```
In [20]: gender = []
val = loan_data.Gender.mode()[0]

for i in loan_data.Gender:
    if pd.isna(i):      # correct way to check missing values
        gender.append(val)
    else:
        gender.append(i)

loan_data['Gender'] = gender
```

```
In [21]: loan_data.LoanAmount=loan_data.LoanAmount.fillna(loan_data.LoanAmount.mean())
```



Replacing categorical null values with mode and continous null values with mean



• Check Null values

```
In [22]: loan_data.isnull().sum()
```

```
Out[22]: Loan_ID          0
Gender              0
ApplicantIncome    0
LoanAmount         0
Loan_Status        0
dtype: int64
```

• Check NA values

```
In [23]: loan_data.isna().sum()
```

```
Out[23]: Loan_ID          0
Gender              0
ApplicantIncome    0
LoanAmount         0
Loan_Status        0
dtype: int64
```



Handled the null values

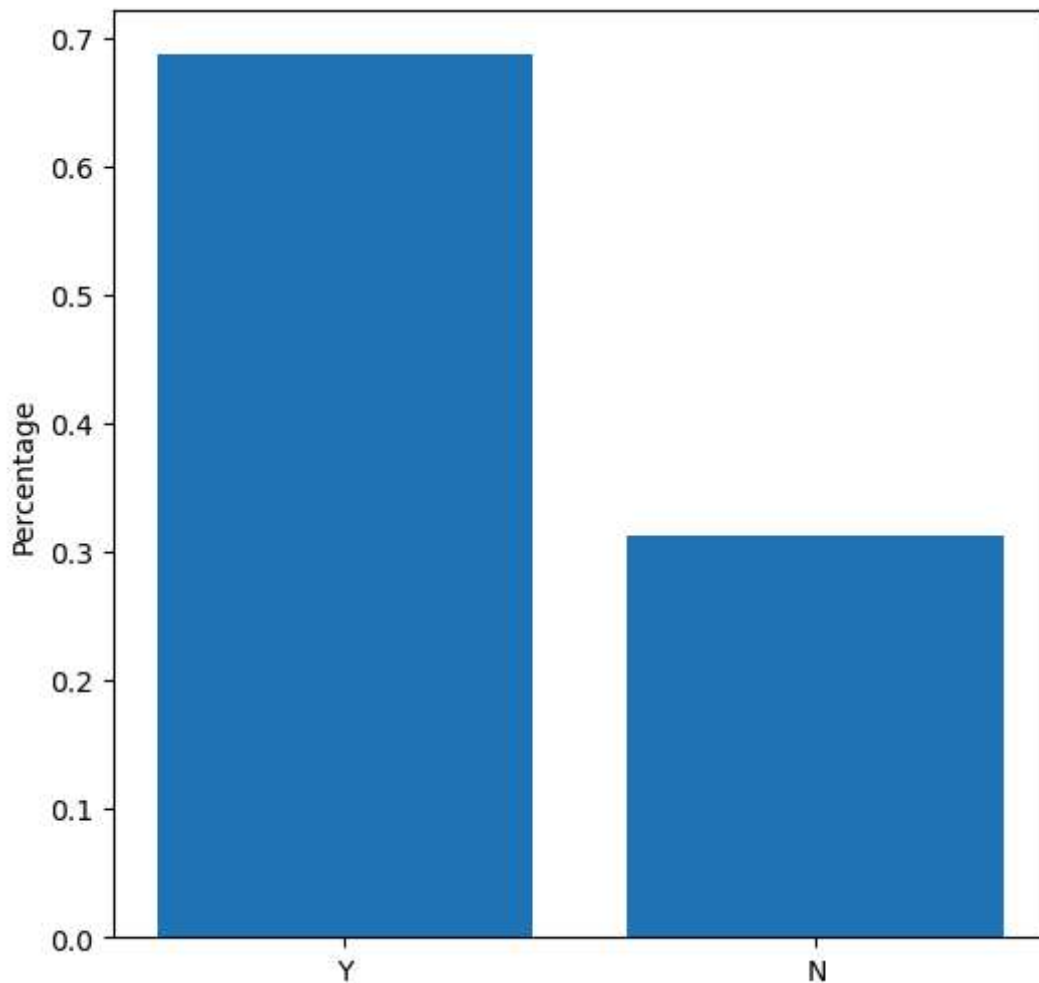


• Check Distribution Of Target Variable

```
In [24]: Target_ratio=loan_data.Loan_Status.value_counts()/len(loan_data)
print(Target_ratio)
```

```
Loan_Status
Y    0.687296
N    0.312704
Name: count, dtype: float64
```

```
In [25]: ## Checking for data unbalance
plt.figure(figsize = (6,6))
plt.bar(Target_ratio.index,Target_ratio)
plt.ylabel('Percentage')
plt.show()
```



Dataset has around 31% of entries where loan application was not accepted.



• Encode Target Variable

```
In [26]: #Using LabelEncoder to encode the target variable
lb=LabelEncoder()
lb.fit(loan_data.Loan_Status)
loan_data.Loan_Status=lb.transform(loan_data.Loan_Status)
```

• Training And Testing Data

```
In [27]: #Divide the dataset into independent and dependent variables
X=loan_data[['ApplicantIncome','LoanAmount']]
y=loan_data.Loan_Status
```

```
In [28]: scl=RobustScaler()
X=scl.fit_transform(X)
```

```
In [29]: #Split the data into training and testing set
X_train,X_test,y_train,y_test=train_test_split(X,y,random_state=0,
                                                test_size=0.4) #40 percent of data will be
```

Support Vector Machine

Some important hyperparameters that should be considered before training the model:

- **C**: Regularization parameter which a float value and is **1.0** by default. Must be strictly positive.
- **kernel**: Specifies the kernel type to be used in the algorithm. 'linear', 'poly', 'rbf'
- **degree**: An optional integer value to specify the degree of polynomial kernel function which is ignored by all other kernels.
- **gamma**: Kernel coefficient for 'rbf', 'poly'
- **coef0**: Independent term in kernel function which is significant in 'poly'.

For further information about other parameter you can refer to: [SVC scikit-learn](#)

• Linear SVM

```
In [30]: svc_loan=SVC(C=10,kernel='linear')
```

```
In [31]: svc_loan.fit(X_train,y_train)
```

```
Out[31]: SVC
SVC(C=10, kernel='linear')
```

Accuracy Score

```
In [32]: pred=svc_loan.predict(X_test)
acc_svc_loan=accuracy_score(y_test,pred)
print('Accuracy Score: ',acc_svc_loan)
```

Accuracy Score: 0.6951219512195121

• Gaussian RBF

```
In [33]: svc_loan=SVC(C=10,kernel='rbf',gamma=2)
```

```
In [34]: svc_loan.fit(X_train,y_train)
```

```
Out[34]: SVC
SVC(C=10, gamma=2)
```

Accuracy Score

```
In [35]: pred=svc_loan.predict(X_test)
acc_svc_loan=accuracy_score(y_test,pred)
print('Accuracy Score: ',acc_svc_loan)
```

Accuracy Score: 0.6788617886178862

Conclusion: In this demonstration, we have gained an understanding of how to apply SVM algorithm.